

Overview of the NTCIR-19 AgenticInstruction Task

Hideo Joho
University of Tsukuba
Tsukuba, Japan
hideo@slis.tsukuba.ac.jp

ABSTRACT

This paper describes the overview of the NTCIR-19 AgenticInstruction task. This pilot task aims to identify effective and robust instructions for guiding Large Language Models (LLMs) in search agent roles. Participants were invited to submit instructions and generation methods covering four key search stages: query formulation, document selection (clicks), relevance judgement, and query reformulation. AgenticInstruction-1 centred on a high-recall ad hoc retrieval scenario in both Japanese and English. Submitted instructions were evaluated in an iterative search setting with performance measured at the session level: the number of unique relevant documents identified (recall) against the number of tokens consumed by the LLM. The task attracted [N] teams which submitted a total of [M] runs. In this paper we describe the task design, the evaluation framework, datasets, organiser baselines, and the results of submitted runs with meta analyses.

KEYWORDS

agentic search, instruction design, large language models, evaluation, high-recall retrieval, test collections

1 INTRODUCTION

Large Language Models (LLMs) are increasingly deployed as autonomous search agents that formulate queries, examine retrieved documents, judge their relevance, and reformulate queries over multiple iterations. In such systems, the natural-language *instructions* that direct each stage of the search process have become a decisive factor in overall performance. However, no systematic comparison of instruction designs exists for search agents: prior work has largely examined single search stages in isolation, whereas real search is multi-stage and iterative. The AgenticInstruction task evaluates instructions in full search sessions, making the *instruction* — rather than the retrieval system — the object of submission and comparison.

The task addresses the following research questions:

- **RQ1a:** What are the best practices for instruction design in agentic search?
- **RQ1b:** What characterises effective and robust instructions for query formulation, document selection, relevance judgement, and query reformulation?
- **RQ1c:** How does instruction performance evolve over multiple iterations in the search process?
- **RQ2a:** What is the relationship between instruction design features and ranking models?
- **RQ2b:** What is the relationship between instruction design features and the choice of LLMs?
- **RQ2c:** To what extent do search topics influence instruction performance?

As a pilot task, AgenticInstruction-1 did not take up RQ2b at the official-run level: a single LLM was fixed for all submitted runs to maximise comparability across participants (Section 3), and the influence of LLM choice was delegated to additional analyses in participant papers.

The remainder of this paper is organised as follows. Section 2 presents the task design, including changes made since the initial design presented at the NTCIR-19 kick-off event. Section 3 describes the evaluation framework. Section 4 introduces the datasets. Section 5 reports the organiser baselines. Section 6 presents the participation and results of submitted runs, and Section 7 provides meta analyses. Section 8 concludes the paper.

2 TASK DESIGN

2.1 Search Task

AgenticInstruction-1 employs a *high-recall ad hoc retrieval* scenario: find as many different relevant documents as possible for a given search topic from a given document collection using a provided search tool. An LLM-based search agent performs a session of up to five *iterations*, where each iteration consists of four stages:

- (1) **Query formulation** (first iteration) or **query reformulation** (subsequent iterations): the LLM produces a query string;
- (2) **Ranking**: the search tool returns a result page (SERP) of ten documents;
- (3) **Document selection (click)**: the LLM selects the documents whose full text should be examined, or none;
- (4) **Relevance judgement**: the LLM judges each selected document as relevant or not relevant based on its full text.

The goal of the task is to develop instruction methods for an LLM to *maximise the total number of unique relevant documents identified (recall)* within five iterations, *while minimising the total number of tokens* sent to and produced by the LLM. A document counts as relevant for evaluation only if it satisfies all three of the following conditions: (1) it was clicked (selected) from a SERP by the LLM; (2) it was judged as relevant by the LLM; and (3) it is labelled as relevant by the official grels.

2.2 Instructions as Submissions

Participants design the instruction text given to the LLM at each stage. Figure 1 shows the instruction template for the query formulation stage: the participant-supplied instruction is combined with fixed descriptions of the task, corpus, and search tool, and the topic description. Customisable parameters include the per-stage instructions, the system prompt, the topic fields presented to the model (title, description, narrative), and the search tool description. The task and corpus descriptions are fixed for submitted runs.

```

Instruction: {participant instruction}
=====
Task Description: {task description}
Corpus Description: {corpus description}
Search Tool Description: {tool description}
Topic Description:
- Title: {title}
- Description: {description}
- Narrative: {narrative}

```

Figure 1: Instruction template for the query formulation stage.

2.3 Run Types

Following NTCIR conventions, two run types were accepted:

- **Automatic run:** any training or human intervention is allowed on the train/dev datasets, but no human intervention is allowed on the test datasets; the instruction method must be frozen before the test datasets are used.
- **Manual run:** human intervention is allowed on the test datasets as well.

Each group could submit up to three runs, where one run consists of session log files for at least two configurations (BM25 and Sparse Encoder) in the participant’s choice of language(s).

2.4 Changes from the Initial Design

Several aspects of the task were revised between the kick-off event (September 2025) and the formal run period, primarily to reduce technical barriers and to maximise comparability across participants:

- **Distribution:** a planned Docker image was cancelled due to technical and licensing issues; instead, the organisers hosted a shared OpenSearch service with per-group credentials, and provided step-by-step environment setup instructions.
- **LLMs:** the initial design listed multiple open-weight models (gpt-oss, Llama, Qwen); the official runs were fixed to a single model, gpt-oss-120b, accessed through Amazon Bedrock. Participants were welcome to test other LLMs in additional analyses.
- **IR models:** DPR was dropped from the official configuration, leaving BM25 and a sparse encoder (SPLADE-style) as the two official ranking models.
- **LLM access:** the task initially adopted a free-tier LLM API service (Groq); as free-tier token limits proved insufficient for full sessions and new paid-tier registrations were closed, the official access path was switched to Amazon Bedrock.

3 EVALUATION FRAMEWORK

3.1 geniie-lab

All experiments are conducted with *geniie-lab*¹, an open-source controlled experimentation testbed for model search behaviour, which extends the experimental framework of Joho and Jose [1].

¹<https://github.com/geniie-lab/geniie-lab>

The testbed instructs an LLM to perform search actions (querying, clicking, judging relevance, reformulating) across configurable, modular stages, records every stage output as structured JSONL logs, and computes session-level statistics. Participants run their instruction methods locally with *geniie-lab* and submit the generated log files; the submitted logs are the run.

3.2 Fixed Components

For submitted runs, the following components were fixed:

- **LLM:** gpt-oss-120b (open-weight, 117B-parameter mixture-of-experts model) accessed via Amazon Bedrock, with `temperature = 0.0` and `top_p = 1.0`;
- **Search tools:** OpenSearch with (a) BM25 ranking (with language-appropriate analysis, e.g. Kuromoji for Japanese) and (b) a multilingual learned sparse encoder (`opensearch-neural-sparse`) over passage-chunked documents, hosted by the organisers;
- **SERP size:** ten results per query;
- **Session plan:** five iterations of the four-stage sequence.

3.3 Measures

The primary measures are computed per topic from the session logs:

- **Unique relevant documents:** the number of distinct documents satisfying the triple condition (clicked, LLM-judged relevant, and qrels-relevant), accumulated across iterations;
- **Recall:** unique relevant documents divided by all relevant documents in the official qrels for the topic;
- **Total tokens:** the sum of tokens sent to and produced by the LLM across all stages of the session;
- **Per-iteration progression:** the cumulative unique relevant documents at the end of each iteration, showing where improvements level off.

Runs are compared primarily on a *recall/token* basis: for a given token budget, higher recall is better; for a given recall, fewer tokens are better.

3.4 Reproducibility Considerations

Although all official runs use greedy decoding (temperature 0), large hosted models are not exactly reproducible: serving-side effects (batching, mixture-of-experts routing, kernel non-determinism) cause small run-to-run variations that compound over a 20-stage session. In repeated organiser baseline runs, 50-topic mean recall varied by roughly ± 0.015 between identical configurations. This inter-run variance constitutes a noise floor against which differences between instruction methods should be interpreted.

4 DATASETS

Table 1 summarises the datasets. English uses newswire collections from TREC Robust; Japanese uses academic abstract collections from NTCIR-1 and NTCIR-2. Train/dev datasets were available throughout the task period; participants were expected to obtain test datasets only when they decided to submit a run, and all development had to be carried out on the train/dev datasets. For the Japanese qrels, both grades “relevant” and “partially relevant” count as relevant (qrels label ≥ 1).

Table 1: Datasets of the AgenticInstruction task.

Language	Role	Dataset	Topics
English	Train/Dev	TREC Robust 2004 (folds 1–2)	100
English	Test	TREC Robust 2005	50
Japanese	Train/Dev	NTCIR-1 AdHoc	83
Japanese	Test	NTCIR-2 AdHoc	50

Table 2: Organiser baselines (default instructions, five iterations, gpt-oss-120b via Amazon Bedrock).

Dataset	IR model	Mean uniq. rel. docs	Mean recall	Total tokens
Robust 2004 fold1	BM25	4.44	0.127	34.0M
Robust 2004 fold1	Sparse	4.98	0.154	44.2M
Robust 2004 fold2	BM25	4.52	0.134	30.2M
Robust 2004 fold2	Sparse	3.86	0.118	30.2M
NTCIR-1 AdHoc	BM25	6.45	0.239	51.0M
NTCIR-1 AdHoc	Sparse	4.93	0.188	40.8M

Documents were indexed by the organisers into per-collection OpenSearch indexes: a BM25 index with language-appropriate analyzers, and a sparse index in which documents are chunked into passages server-side and each passage is encoded into a learned sparse representation; at query time, a document is scored by its best-matching passage. Queries are encoded on the participant side with the same multilingual sparse encoder, so no model-serving permissions are required of participants.

5 ORGANISER BASELINES

To provide participants with a reference point, the organisers produced baselines for all six train/dev configurations using the default instructions distributed with the task (e.g. “Review the provided descriptions of task, corpus, tool and search topic. Then, formulate a search query.”), with no optimisation. Table 2 summarises the results; the full per-topic tables are published on the task website.²

Three observations characterise the default-instruction behaviour. First, *improvements concentrate in early iterations*: roughly 75–80% of final recall is obtained in the first iteration, and gains largely level off by the third iteration, while later iterations are the most token-expensive because the conversation history is at its largest. Second, in English the two IR models perform comparably under default instructions, whereas in Japanese BM25 clearly outperforms the multilingual sparse encoder (mean recall 0.239 vs. 0.188). Third, session cost is dominated by clicking behaviour: the default instructions click generously (often most of the SERP), so a full session consumes roughly 0.6–0.9M tokens per topic. Together these define the challenge posed to participants: make later iterations productive, or stop early and save tokens.

²<https://genie-lab.github.io/ntcir/>

6 PARTICIPATION AND RESULTS

[To be completed after run submissions close on 10 August 2026.]

7 META ANALYSES

[To be completed after run submissions and participant paper drafts.]

8 CONCLUSIONS AND FUTURE DIRECTIONS

This paper presented an overview of the NTCIR-19 AgenticInstruction task, a pilot task for identifying effective and robust instructions for LLM-based search agents, evaluated at the session level in a high-recall ad hoc scenario across English and Japanese.

Several directions are candidates for a future cycle: graded relevance in the session evaluation; additional languages and genres; letting the agent plan its own session (agentic plans) rather than following a fixed iteration structure; instruction robustness protocols across serving stacks; and multi-LLM comparisons (RQ2b), which this pilot deliberately left to participant analyses.

ACKNOWLEDGMENTS

REFERENCES

- [1] Hideo Joho and Joemon M. Jose. 2025. TODO: exact title of the SIGIR 2025 paper that genie-lab extends. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2025)*. TODO: verify title, pages, DOI.